African Scientific Research and Innovation Council

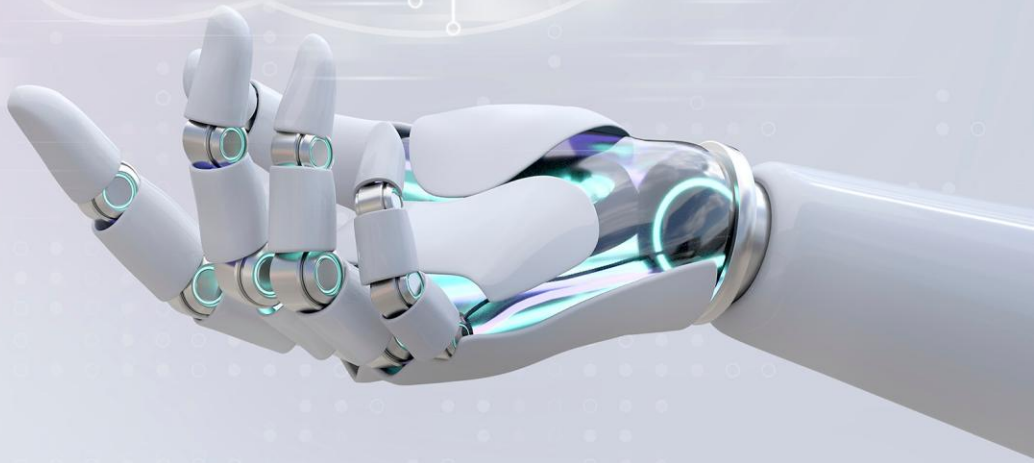
BAAI 智源研究院

北京大学 PEKING UNIVERSITY

中国人民大学 RENMIN UNIVERSITY OF CHINA

Global AI Compute Faculty Development Initiative

Module 03: AI Compiler Principles and Optimization



Guojie Luo gluo@pku.edu.cn

Jing Wang jwang@ruc.edu.cn

October 2026 · 2nd AICFD

Supported by UNDP

Review: Programming with Triton

- More examples…
- FlagGems (to register with the ATen backend of PyTorch)
  - https://github.com/flagos-ai/FlagGems/tree/master/src/flag_gems/ops
- Triton kernels in SGLang
  - https://github.com/sgl-project/sglang/tree/main/python/sglang/kernels/ops

Language Processing Systems

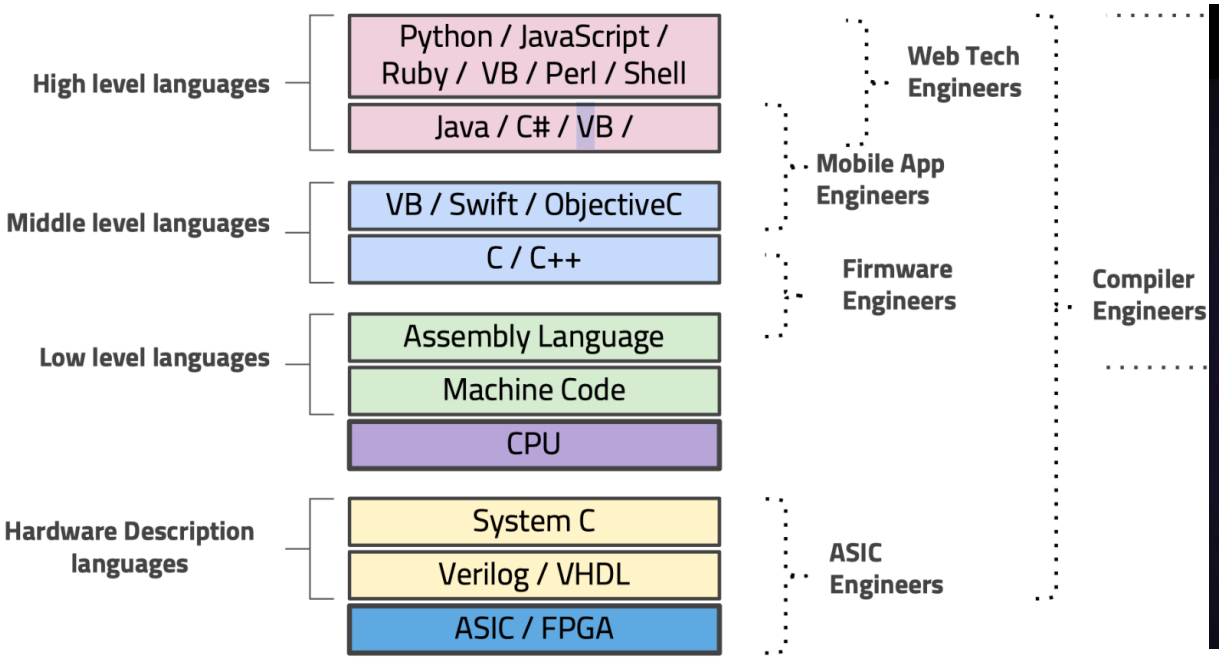
➢Role of a language-processing system

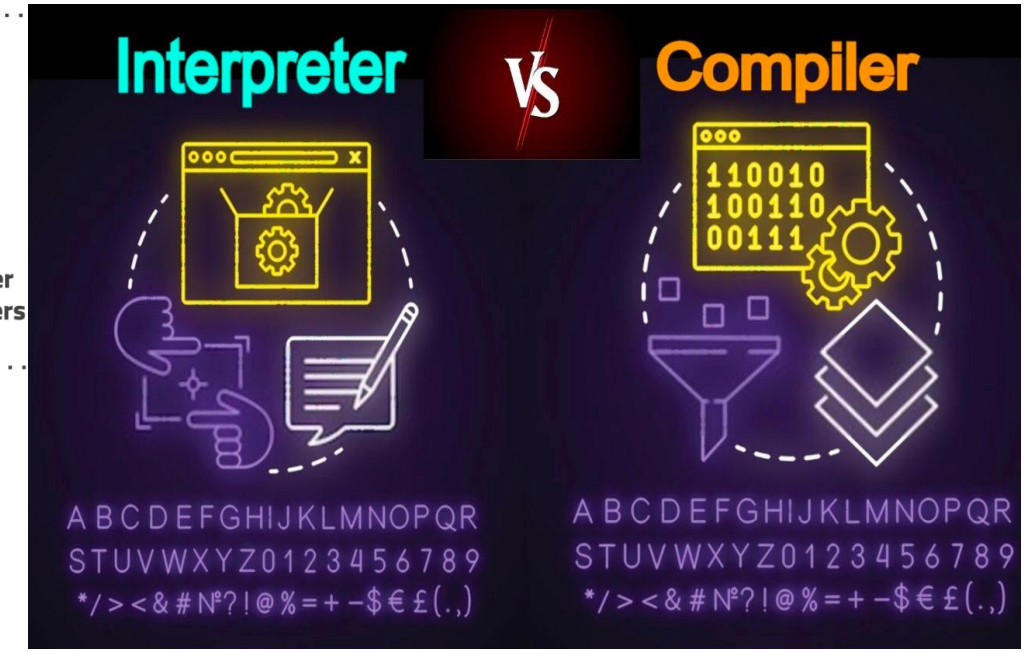
Convert source programs into executable code, results, or intermediate forms.

➢Language-processing approaches

Compilation-based systems

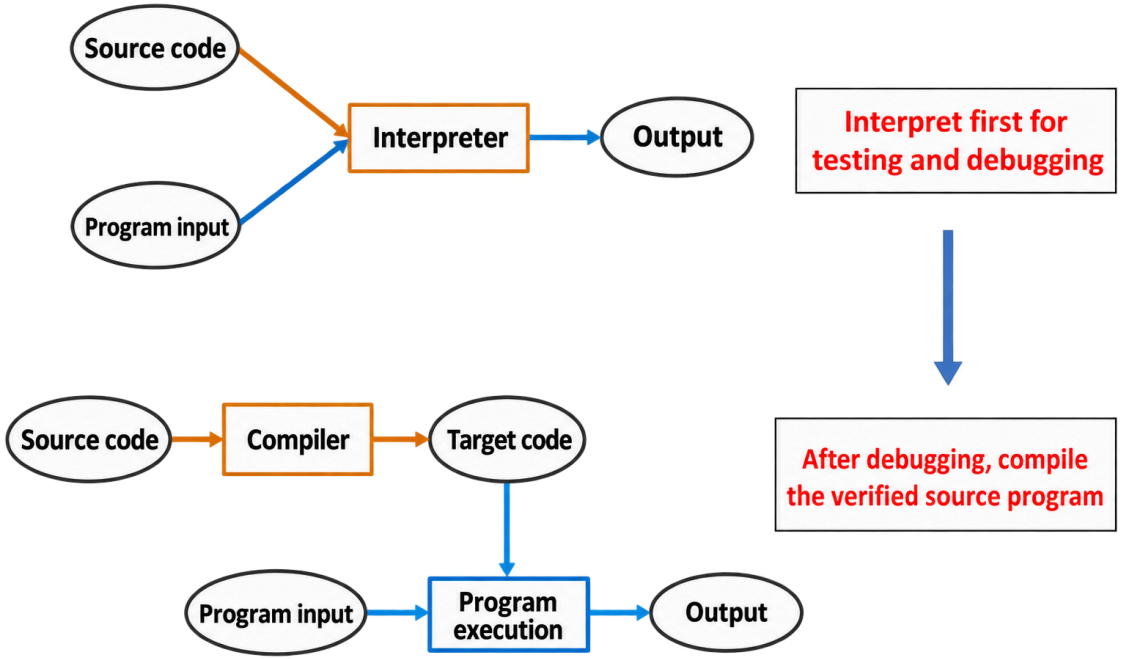
Interpretation-based systems

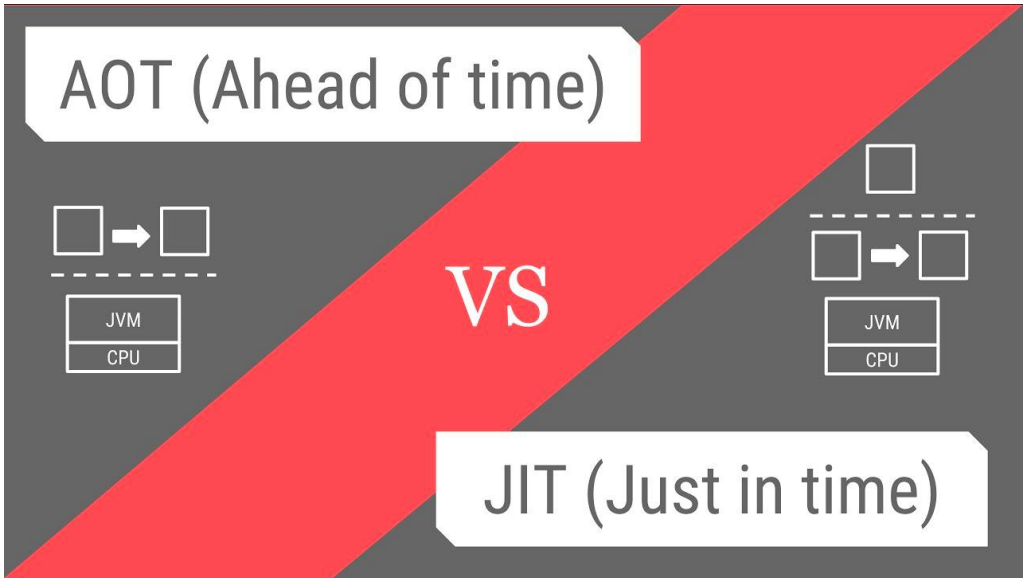




AOT and JIT Compilation

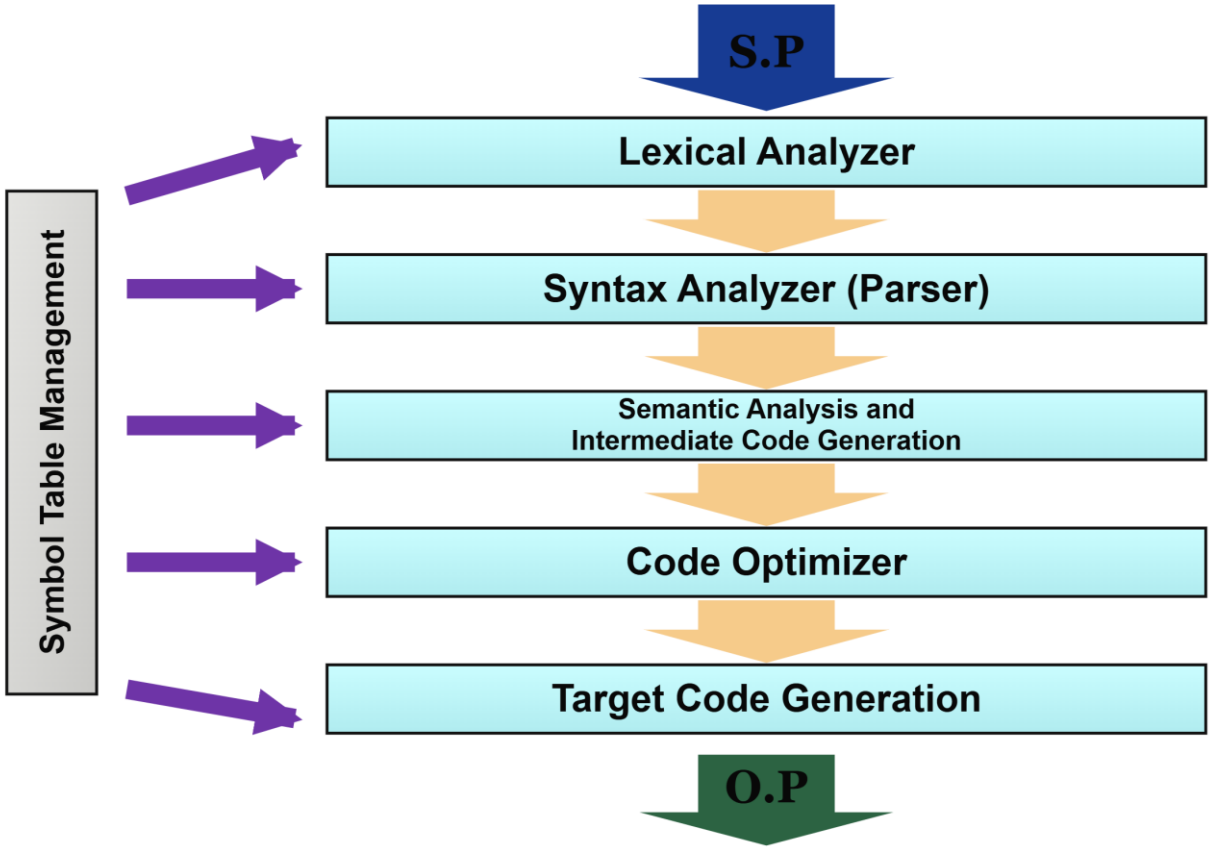
1. AOT (ahead-of-time): compile before execution. Example: MLIR + TensorFlow.
2. JIT (just-in-time): compile during execution. Examples: PyTorch and Tsinghua Jittor.





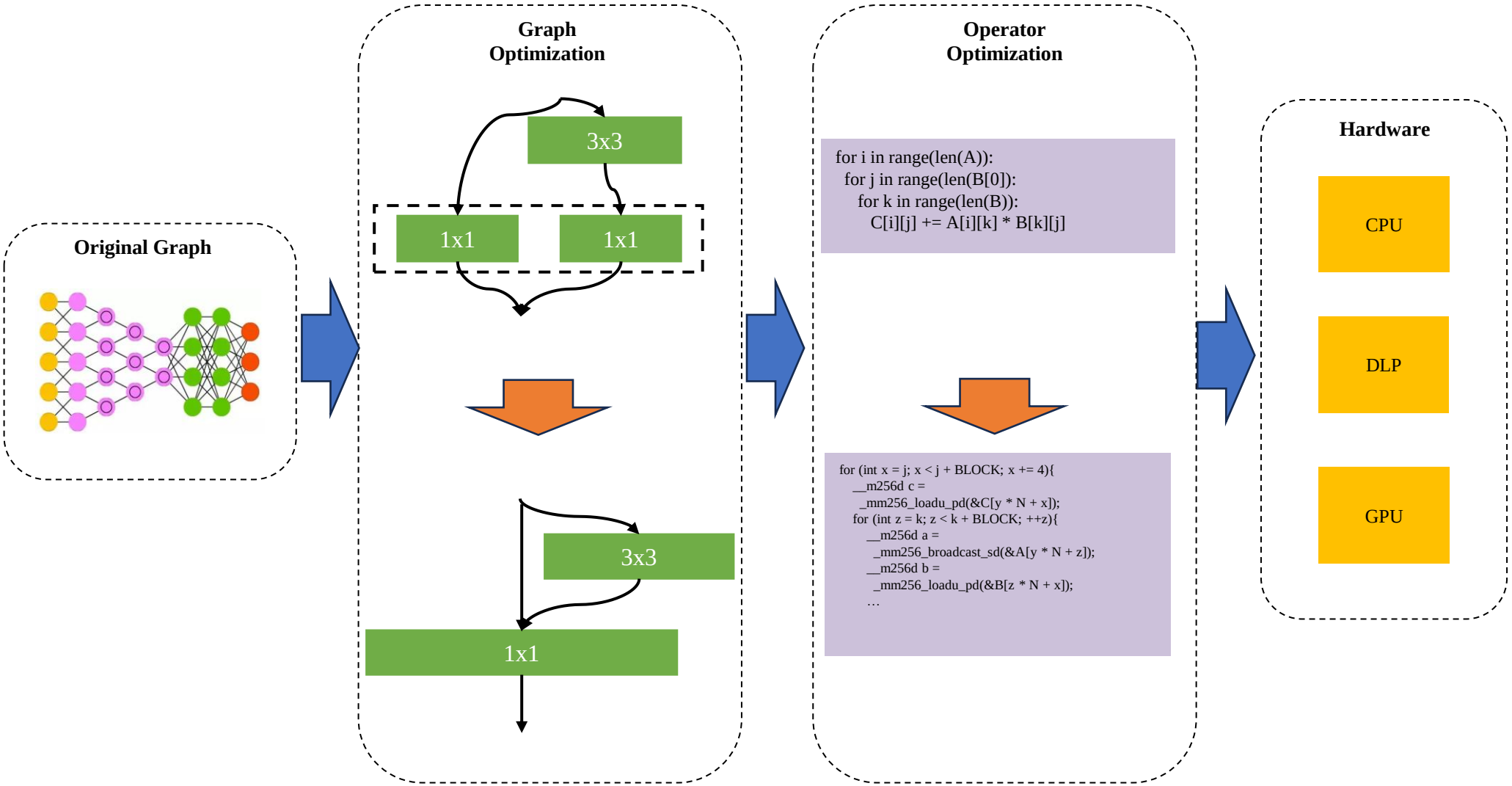
Compilation Flow

Seven Logical Components




```python
def build_vggnet(vgg19_npy_path):
  models = {}
  models['input'] = tf.Variable(np.zeros((1, 224, 224, 3)).astype('float32'))
  models['conv1_1'] = basic_calc('conv',models['input'],read_wb(vgg19_npy_path,'conv1_1'))
  models['conv1_2'] = basic_calc('conv',models['conv1_1'],read_wb(vgg19_npy_path,'conv1_2'))
  models['pool1']   = basic_calc('pool',models['conv1_2'])
  models['conv2_1'] = basic_calc('conv',models['pool1'],read_wb(vgg19_npy_path,'conv2_1'))
  models['conv2_2'] = basic_calc('conv',models['conv2_1'],read_wb(vgg19_npy_path,'conv2_2'))
  models['pool2']   = basic_calc('pool',models['conv2_2'])
  models['conv3_1'] = basic_calc('conv',models['pool2'],read_wb(vgg19_npy_path,'conv3_1'))
  models['conv3_2'] = basic_calc('conv',models['conv3_1'],read_wb(vgg19_npy_path,'conv3_2'))
  models['conv3_3'] = basic_calc('conv',models['conv3_2'],read_wb(vgg19_npy_path,'conv3_3'))
  models['conv3_4'] = basic_calc('conv',models['conv3_3'],read_wb(vgg19_npy_path,'conv3_4'))
  models['pool3']   = basic_calc('pool',models['conv3_4'])
  models['conv4_1'] = basic_calc('conv',models['pool3'],read_wb(vgg19_npy_path,'conv4_1'))
  models['conv4_2'] = basic_calc('conv',models['conv4_1'],read_wb(vgg19_npy_path,'conv4_2'))
  models['conv4_3'] = basic_calc('conv',models['conv4_2'],read_wb(vgg19_npy_path,'conv4_3'))
  models['conv4_4'] = basic_calc('conv',models['conv4_3'],read_wb(vgg19_npy_path,'conv4_4'))
  models['pool4']   = basic_calc('pool',models['conv4_4'])
  models['conv5_1'] = basic_calc('conv',models['pool4'],read_wb(vgg19_npy_path,'conv5_1'))
  models['conv5_2'] = basic_calc('conv',models['conv5_1'],read_wb(vgg19_npy_path,'conv5_2'))
  models['conv5_3'] = basic_calc('conv',models['conv5_2'],read_wb(vgg19_npy_path,'conv5_3'))
  models['conv5_4'] = basic_calc('conv',models['conv5_3'],read_wb(vgg19_npy_path,'conv5_4'))
  models['pool5']   = basic_calc('pool',models['conv5_4'])
  return models
```

What Is an AI Compiler?



AI Compilers

Frontend: Python-based DSLs

e.g., PyTorch and TensorFlow

Backend: AI accelerators

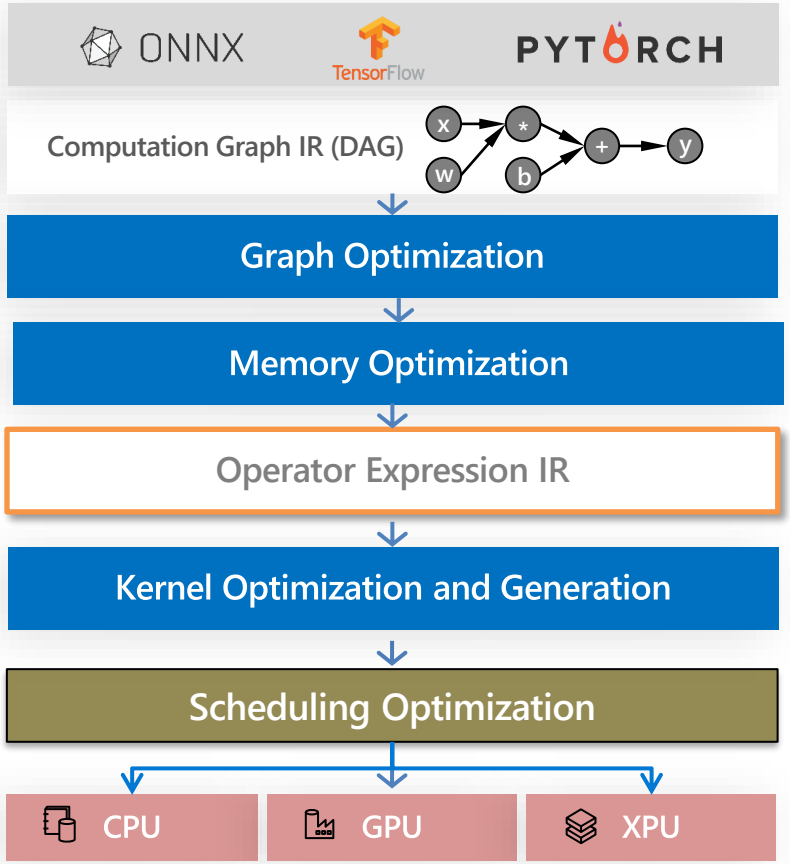
e.g., GPU, CPU, FPGA, TPU, IPU

IR: graphs and operator expressions

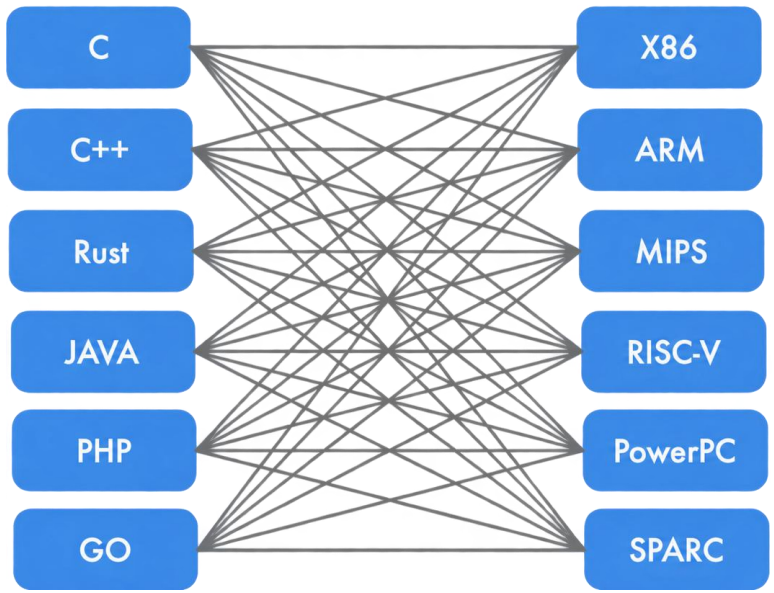
e.g., DAGs and TVM IR

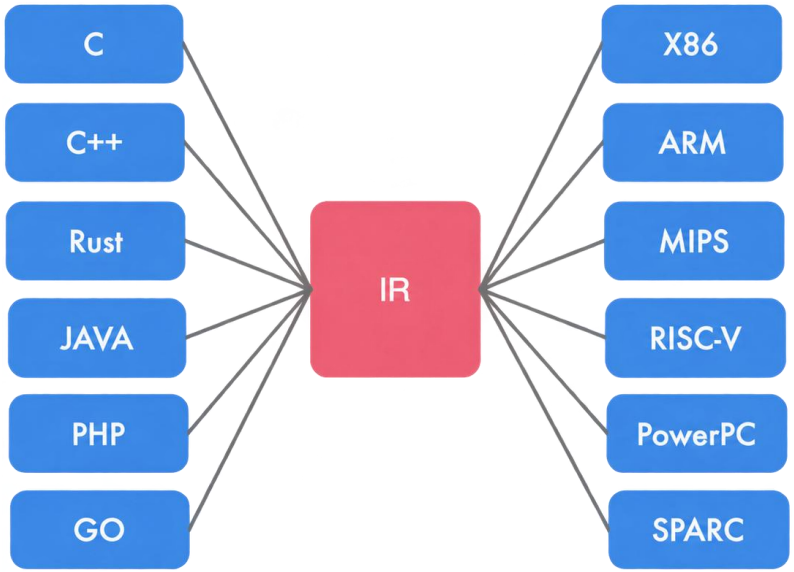
Optimizations:

Graphs, memory, kernels, and scheduling



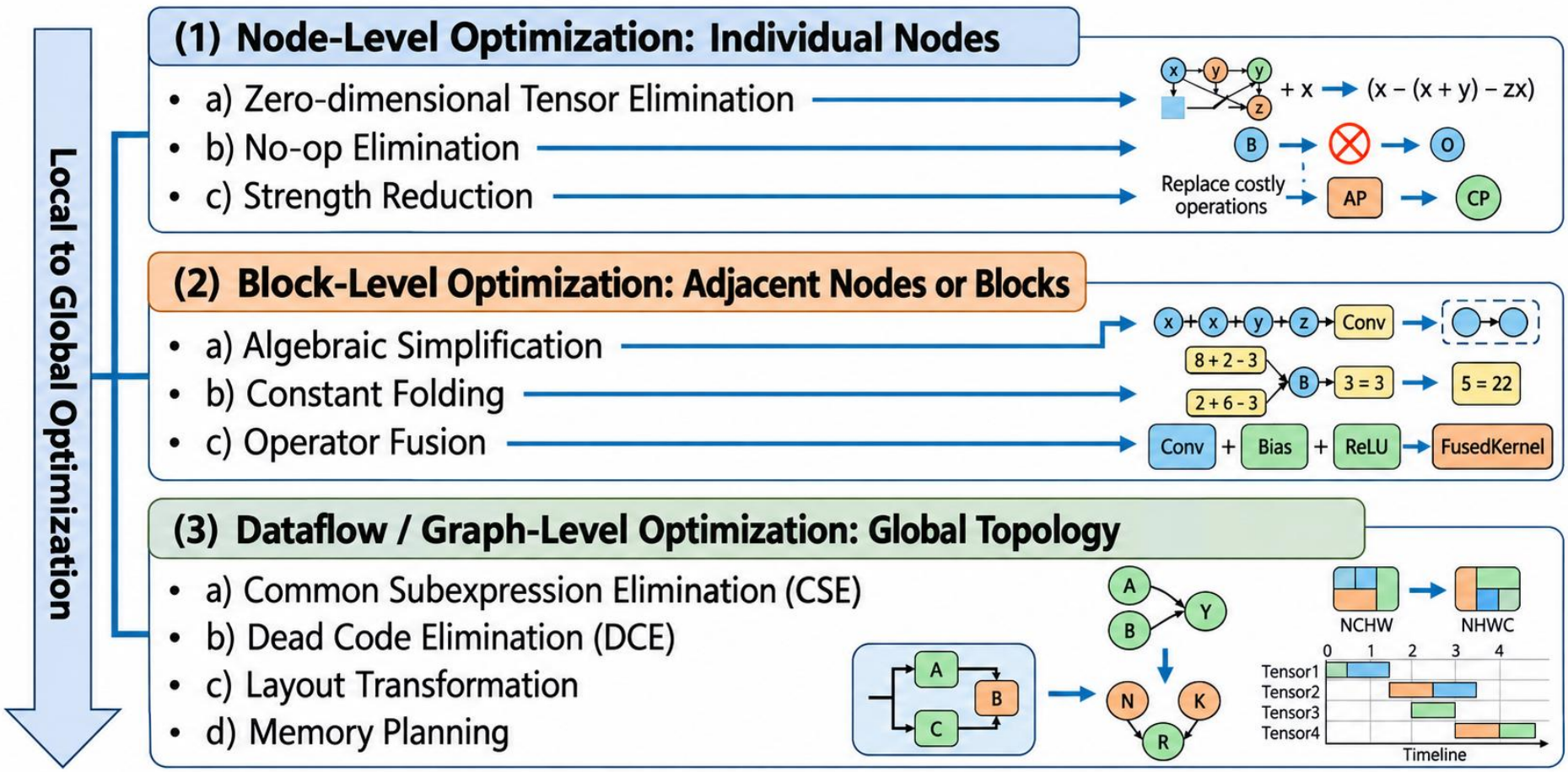
GCC vs LLVM


C
C++
Rust
JAVA
PHP
GO
X86
ARM
MIPS
RISC-V
PowerPC
SPARC


C
C++
Rust
JAVA
PHP
GO
IR
X86
ARM
MIPS
RISC-V
PowerPC
SPARC

4.3. Compiler-Level Optimization Strategies

Graph Optimization



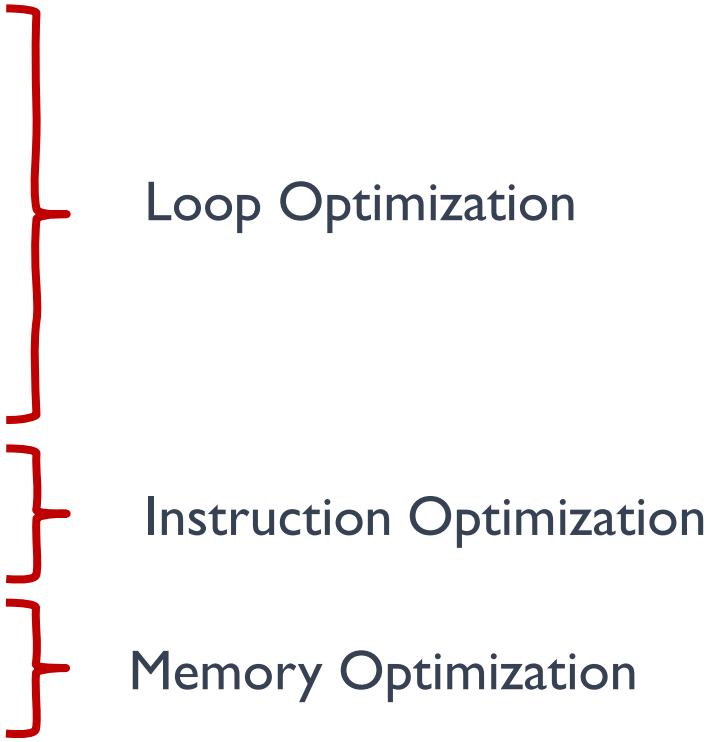
Operator-Level Optimization

Lower optimized graph nodes to operator-level IR.

Operator scheduling: optimize compute and memory access for the target.

Autotuning: search for the best scheduling configuration.

- Loop Unrolling
- Loop Tiling
- Loop Reordering
- Loop Fusion
- Loop Splitting
- Vectorization
- Tensorization
- Latency Hiding
- Memory Allocation



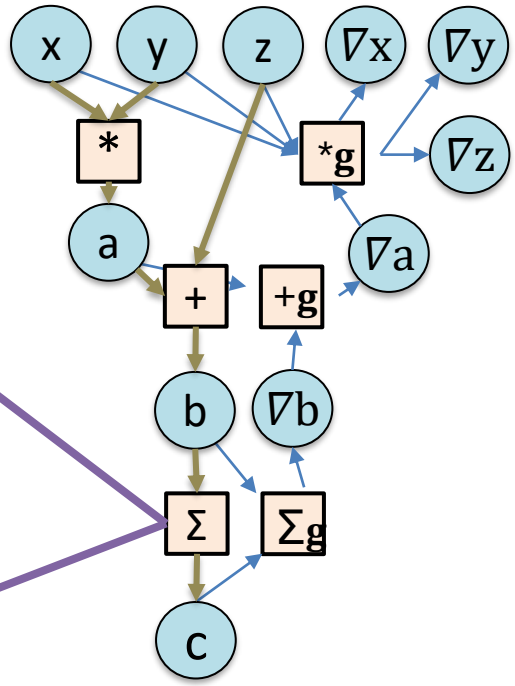
Operator-Level Optimization

Example: CPU/GPU Kernels Registered in TensorFlow

```
1  template <typename Device, typename OUT_T, typename IN_T,
2            typename ReductionAxes, typename Scalar>
3  struct ReduceEigenImpl<Device, OUT_T, IN_T, ReductionAxes,
4                         functor::MeanReducer<Scalar>> {
5    void operator()(const Device& d, OUT_T out, IN_T in,
6                    const ReductionAxes& reduction_axes,
7                    const functor::MeanReducer<Scalar>& reducer) {
8      static_assert(std::is_same<Scalar, typename OUT_T::Scalar>::value, "");
9      Eigen::internal::SumReducer<Scalar> sum_reducer;
10     out.device(d) = in.reduce(reduction_axes, sum_reducer) /
11                     static_cast<Scalar>(in.size() / out.size());
12   }
13 };
14
15 // T: the data type
16 // REDUCER: the reducer functor
17 // NUM_AXES: the number of axes to reduce
18 // IN_DIMS: the number of dimensions of the input tensor
19 #define DEFINE(T, REDUCER, IN_DIMS, NUM_AXES)                          \
20   template void ReduceFunctor<GPUDevice, REDUCER>::Reduce(
21       OpKernelContext* ctx, TTypes<T, IN_DIMS - NUM_AXES>::Tensor out, \
22       TTypes<T, IN_DIMS>::ConstTensor in,                              \
23       const Eigen::array<Index, NUM_AXES>& reduction_axes,             \
24       const REDUCER& reducer);
```

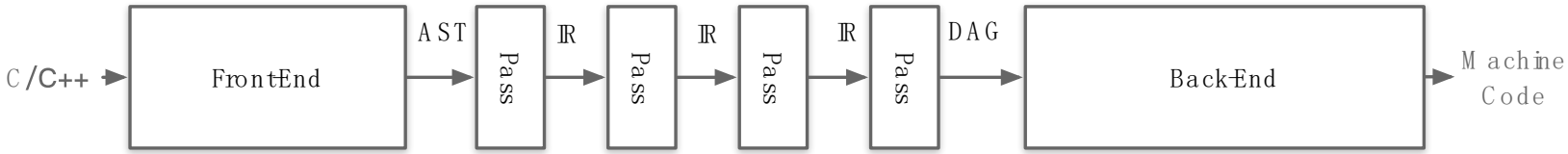
CPU code

GPU code



Compiler Optimization Passes

After IR generation, analysis and transformation passes prepare LLVM IR for target code or assembly generation.




```c
#include <stdio.h>

int loop_add(int* data, int num) {
  int res = 0;
  for (int i = 0; i < num; i ++) {
    res += data[i];
  }
  return res;
}
```

```
define dso_local i32 @loop_add(i32*, i32) #0 {
  %3 = alloca i32*, align 8
  %4 = alloca i32, align 4
  %5 = alloca i32, align 4
  %6 = alloca i32, align 4
  store i32* %0, i32** %3, align 8
  store i32 %1, i32* %4, align 4
  store i32 0, i32* %5, align 4
  store i32 0, i32* %6, align 4
  br label %7

; <label>:7:                                      ; preds = %19, %2
  %8 = load i32, i32* %6, align 4
  %9 = load i32, i32* %4, align 4
  %10 = icmp slt i32 %8, %9
  br i1 %10, label %11, label %22

; <label>:11:                                     ; preds = %7
  %12 = load i32*, i32** %3, align 8
  %13 = load i32, i32* %6, align 4
  %14 = sext i32 %13 to i64
  %15 = getelementptr inbounds i32, i32* %12, i64 %14
  %16 = load i32, i32* %15, align 4
  %17 = load i32, i32* %5, align 4
  %18 = add nsw i32 %17, %16
  store i32 %18, i32* %5, align 4
  br label %19

; <label>:19:                                     ; preds = %11
  %20 = load i32, i32* %6, align 4
  %21 = add nsw i32 %20, 1
  store i32 %21, i32* %6, align 4
  br label %7

; <label>:22:                                     ; preds = %7
  %23 = load i32, i32* %5, align 4
  ret i32 %23
}
```



1. From Triton to Compilation Flow

- General Flow: Compilation Procedure
- General Flow: Computation Graph
- General Flow: Graph Optimization
- General Flow: torch.compile
- General Flow: TorchDynamo Overview
- General Flow: TorchDynamo Example
- General Flow: TorchInductor

1.1. General Flow: Compilation Procedure

- High-level representation lowering to hardware code gradually
  - High-level framerwork: pyTorch, TensorFlow, …
  - High-level DSL: Triton, Tilelang, …
  - Low-level DSL: CUDA, Ascend C, …
  - Intermediate Representations: MLIR(TIR, TGIR), LLVM, …
  - Hardware Code: PTX, CPU ISA, NPU ISA, …
- Introduce hardware information gradually and optimize automatically
  - Computation graph optimziation, parameter sweeping, register assignment, …
- IR/DSL to meet the needs of AI comutation
  - TVM: tensor operation
  - Tilelang: tile-level abstract

1.2a. General Flow: Computation Graph

- Computation graph: Design the dependency among opertors in AI-computing tasks
  - Static graph (Tensorflow1.x)
    - Specify at compilation, easy to optimize
  - Dynamic graph (PyTorch)
    - Specify at runtime, fit for modern LLM
- Why computation graph
  - Represent forward and backward relationship
  - Fit for multi-level AI operator representation
- Components
  - Nodes: Tensor/Data or operation
  - Edges: Opertaion or Tensor/Data

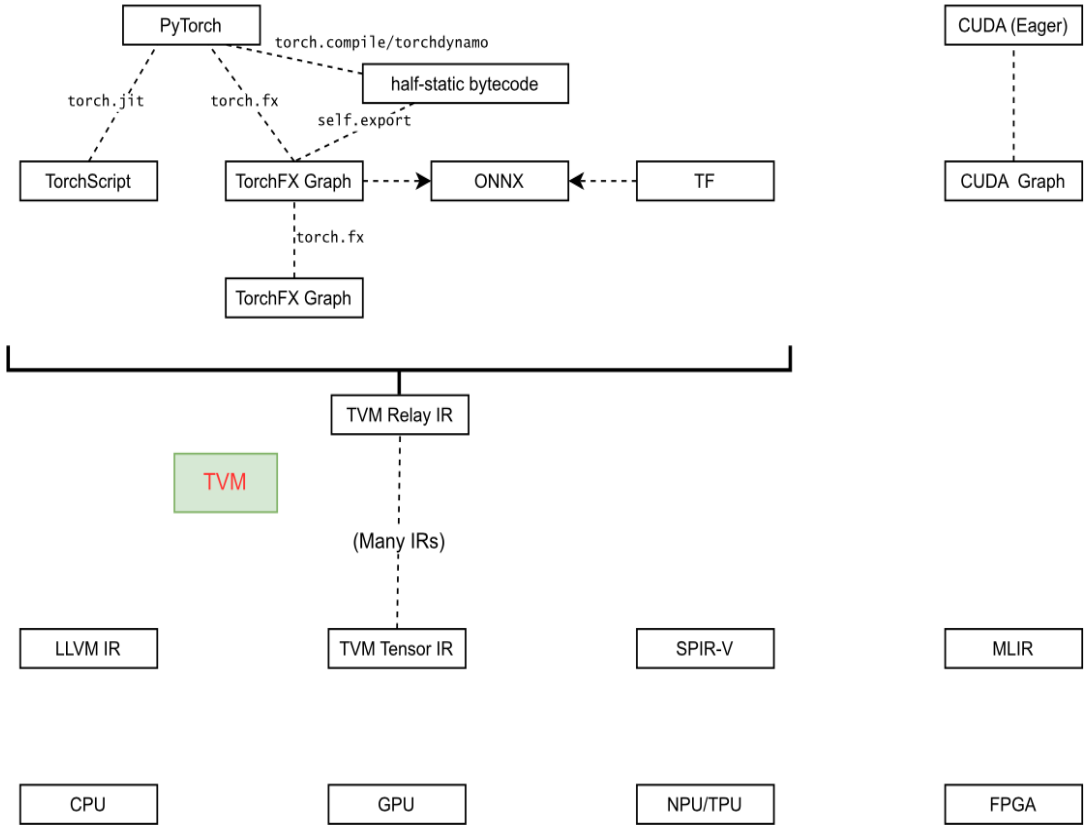


Image Source: https://blog.rijuyuezhu.top/posts/54261af8/

1.2b. General Flow: Computation Graph

- Computation graph construction
  - Static: Define variable and operation before compilation → Obtain computation graph
  - Dynamic
    - Define variables
    - Forward pass
    - Construct on the fly
    - Backward pass
- Optimization time
  - Static: compilation
  - Dynamic: runtime

```python
# Importing torch
import torch

# Initializing input tensors
a = torch.tensor(15.0, requires_grad=True)
b = torch.tensor(20.0, requires_grad=True)

# Computing the output
c = a * b

# Computing the gradients
c.backward()

# Collecting the output gradient of the
# output with respect to the input 'a'
derivative_out_a = a.grad

# Collecting the output gradient of the
# output with respect to the input 'b'
derivative_out_b = b.grad

# Displaying the outputs
print(f'c = {c}')
print(f'Derivative of c with respect to a = {derivative_out_a}')
print(f'Derivative of c with respect to b = {derivative_out_b}')
```

PyTorch Dynamic Graph

```python
# Importing tensorflow version 1
import tensorflow.compat.v1 as tf
tf.disable_v2_behavior()

# Initializing placeholder variables of
# the graph
a = tf.placeholder(tf.float32)
b = tf.placeholder(tf.float32)

# Defining the operation
c = tf.multiply(a, b)

# Instantiating a tensorflow session
with tf.Session() as sess:

    # Computing the output of the graph by giving
    # respective input values
    out = sess.run([c], feed_dict={a: [15.0], b: [20.0]})[0][0]

    # Computing the output gradient of the output with
    # respect to the input 'a'
    derivative_out_a = sess.run(tf.gradients(c, a), feed_dict={
                                a: [15.0], b: [20.0]})[0][0]

    # Computing the output gradient of the output with
    # respect to the input 'b'
    derivative_out_b = sess.run(tf.gradients(c, b), feed_dict={
                                a: [15.0], b: [20.0]})[0][0]

    # Displaying the outputs
    print(f'c = {out}')
    print(f'Derivative of c with respect to a = {derivative_out_a}')
    print(f'Derivative of c with respect to b = {derivative_out_b}')
```

Tensorflow Static Graph

Example Source: https://www.geeksforgeeks.org/deep-learning/dynamic-vs-static-computational-graphs-pytorch-and-tensorflow/

1.3a. General Flow: Graph Optimization

- Operation fusion
  - Reduce data transimission
  - Reduce launch overhead
  - Simplify graph pattern
- Redundant node elimination
  - Certain operations are inactive
    - cast between same data types
- Constant folding
  - Calculate in advance for constants

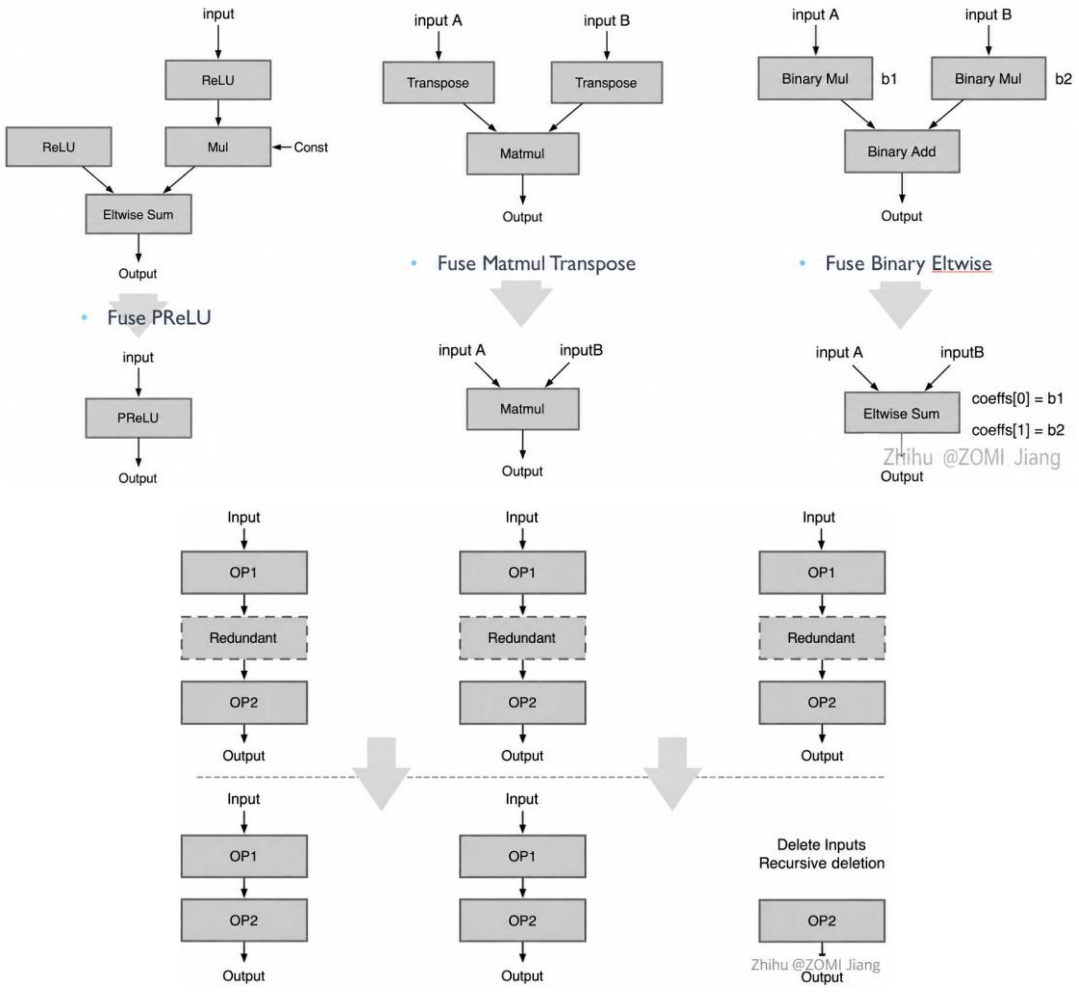
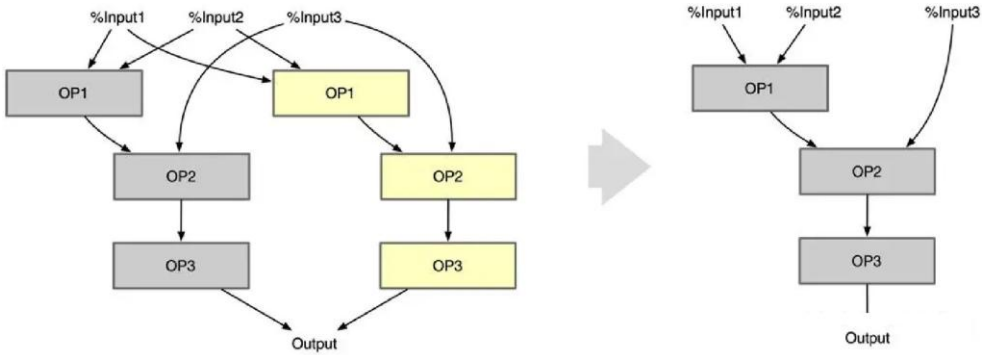


Image Source: https://zhuanlan.zhihu.com/p/7795643474

1.3b. General Flow: Graph Optimization

- Common subgraph optimization
  - Common subgraph: same structure, parameter and input
  - Reuse subgraph



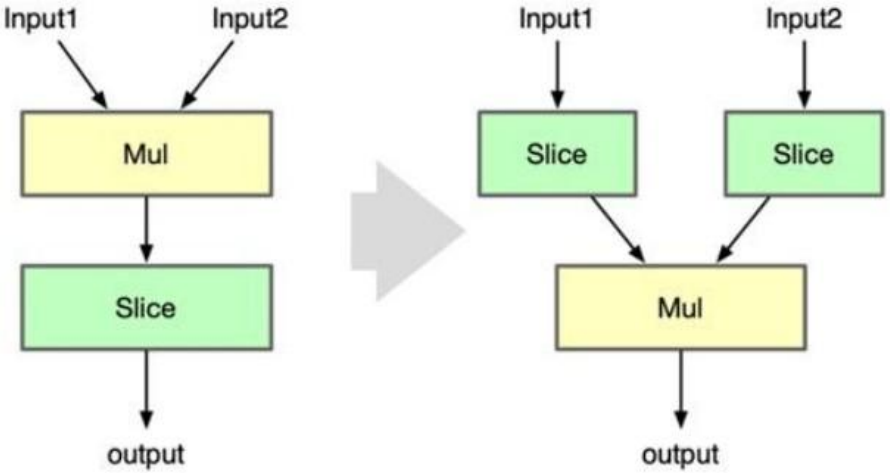
- Operator substitute
  - Equivalent operator subsitution for certain hardware feature
    - Conv can be calculated by TensorCore by adding some additional calculation
    - Two consecutive conv can be subtitute by an equivalent conv

Image Source: https://zhuanlan.zhihu.com/p/7795643474

1.3c. General Flow: Graph Optimization

- Operation scheduling

Caluculate some operation in advance to reduce duplicate computation



Reduce data transfer cost

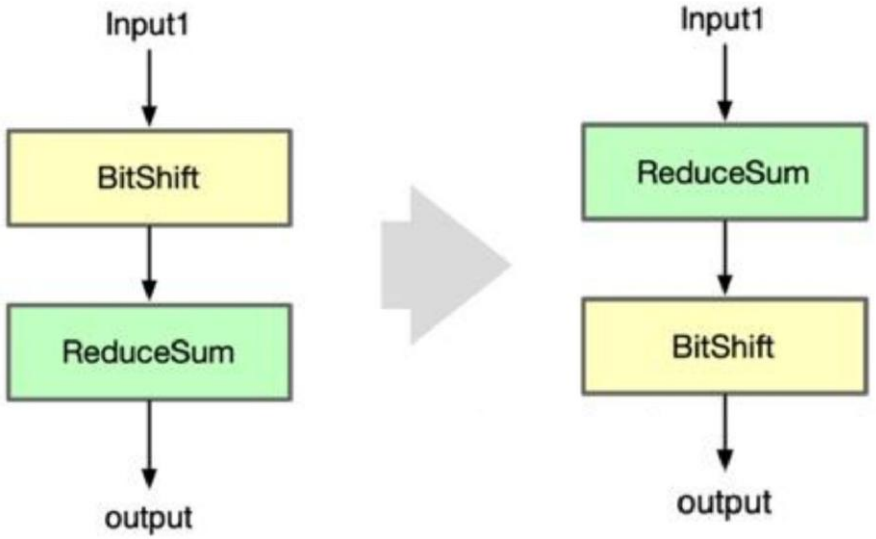


Image Source: https://zhuanlan.zhihu.com/p/7795643474

1.4a. General Flow: torch.compile

- Dymanic-Static Tradeoff
  - Both PyTorch and TensorFlow extends thier computation graph (torch.fx and TensorFlow2.0)
  - Easy to develop and efficiently optimized for execution
- torch.compile: Benchmark and Optimization On-the-fly
  - Main idea: Optimize current computation as long as it does not change
  - Faster Compilation and faster execution after first run

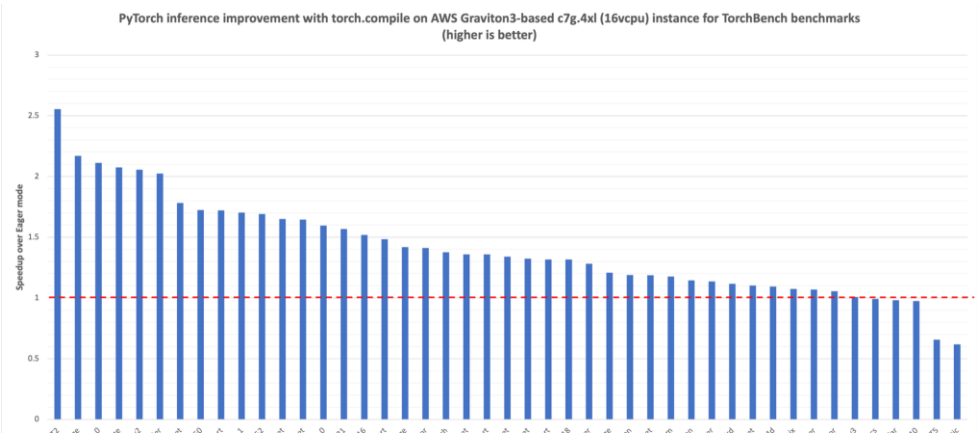


Image Source: https://pytorch.ac.cn/blog/accelerated-pytorch-inference/

1.4b. General Flow: torch.compile

- TorchDynamo
  - Rewrite python bytecode
  - Retrieved computation graph from bytecode
  - Output forward static graph in first run
- AOTAutoGrad
  - AOTDispatcher: dispatch operator to specific kernel
  - AOT-ahead of time: trace BP graph simultaneously as forward computation graph
- TorchInductor
  - Optimize graph and generate hardware code
  - Leverage kernal fusion and other methods for sepcific architecture

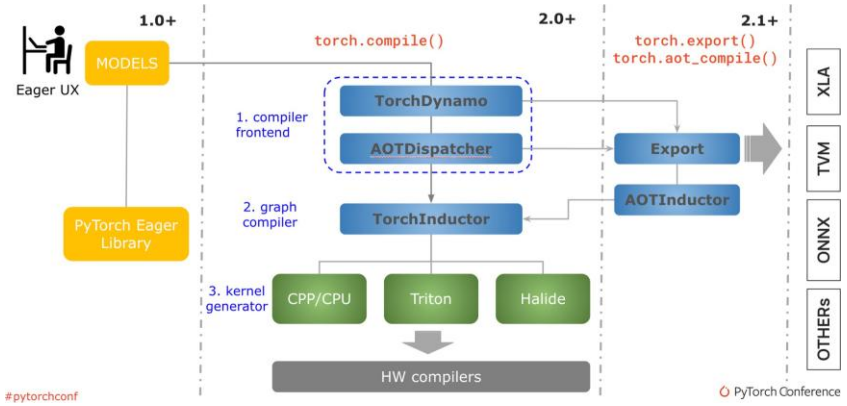


Image Source: https://pytorch.ac.cn/blog/accelerated-pytorch-inference/

1.5. General Flow: TorchDynamo Overview

- Eager Mode Pros and Cons:
  - PyTorch define-by-run mechanism is highly flexible and well-loved by researchers. However, the framework can only see one operator at a time, making cross-operator graph-level optimization (e.g., operator fusion, memory scheduling) difficult.
- Compromises of Previous Approaches:
  - TorchScript: Attempts static parsing of Python AST, facing “all-or-nothing” dilemma, easily failing because it doesn’t support complete Python semantics.
  - Lazy Tensors: High runtime overhead, delaying low-level kernel launch and causing frequent recompilation.
- TorchDynamo Core Motivation:
  - As a Python-level Just-In-Time compiler, the goal is to achieve Python graph compilation without sacrificing any PyTorch flexibility.

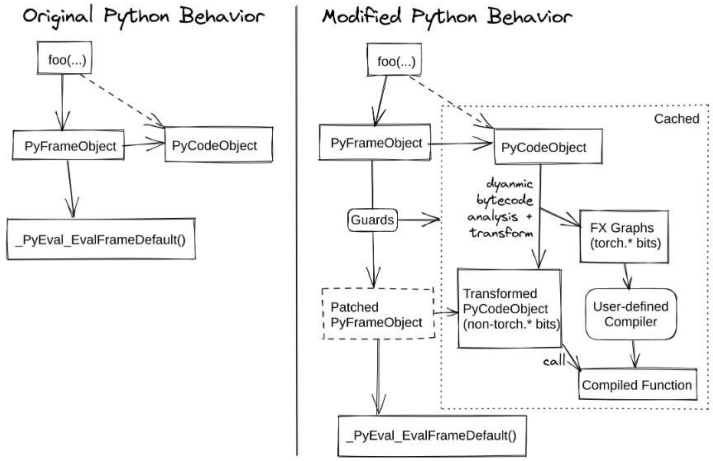


Figure 1. Overview of how TorchDynamo modifies the CPython interpreter to capture FX graphs

Ansel, Jason, et al. “Pytorch 2: Faster machine learning through dynamic python bytecode transformation and graph compilation.” Proceedings of the 29th ACM international conference on architectural support for programming languages and operating systems, volume 2. 2024.

1.6. General Flow: TorchDynamo Example

An Example

```python
from typing import List
import torch
import torchdynamo

def toy_example(a, b):
    x = a / (torch.abs(a) + 1)
    if b.sum() < 0:
        b = b * -1
    return x * b

def my_compiler(gm: torch.fx.GraphModule,
                example_inputs: List[torch.Tensor]):
    print("my_compiler() called with FX graph:")
    gm.graph.print_tabular()
    return gm.forward  # return a python callable

with torchdynamo.optimize(my_compiler):
    for _ in range(100):
        toy_example(torch.randn(10),
                    torch.randn(10))
```

Example Output

```
my_compiler() called with FX graph:
opcode         name     target             args
-------------  -------  -----------------  -----------------
placeholder    a        a                  ()
placeholder    b        b                  ()
call_function  abs_1    torch.abs          (a,)
call_function  add      operator.add       (abs_1, 1)
call_function  truediv  operator.truediv   (a, add)
call_method    sum_1    sum                (b,)
call_function  lt       operator.lt        (sum_1, 0)
output         output   output             ((truediv, lt),)
```

```
my_compiler() called with FX graph:
opcode         name     target             args
-------------  ------   -----------------  -----------
placeholder    b        b                  ()
placeholder    x        x                  ()
call_function  mul      operator.mul       (b, -1)
call_function  mul_1    operator.mul       (x, mul)
output         output   output             ((mul_1,),)
```

```
my_compiler() called with FX graph:
opcode         name     target             args
-------------  ------   -----------------  ---------
placeholder    b        b                  ()
placeholder    x        x                  ()
call_function  mul      operator.mul       (x, b)
output         output   output             ((mul,),)
```

Image Source: https://chips-compilers-mlsys-22.github.io/assets/slides/PyTorch%20Compilers%20(Compiler%20&%20Chips%20Symposium%202022).pdf

1.7. General Flow: TorchInductor

- Graph fusion: Combines multiple pointwise operators
  - May generate equivalent Triton kernels on the fly
- Scheduling: Determines loop ordering, tile sizes, and parallel execution
- Backend Selection
  - GPU → Triton
  - CPU → C++/OpenMP
- Memory planning: reduces temporary tensor allocation

2. IR, Graph Capture, and torch.compile

- AI-DSL Design: Why DSL?
- AI-DSL Design: How to Design DSL
- AI-DSL Design: Triton
- AI-DSL Design: TileLang
- TileLang: Schedule Annotations and Primitives

2.1. AI-DSL Design: Why DSL?

- Gap between AI computing abstract and GPU execution model
  - Deep learning models are naturally expressed as tensor algebra
  - GPUs execute massively parallel scalar instructions
- Human/Agent workload
  - Explicit thread/block management, Memory hierarchy (global, shared, registers), Synchronization primitives, Instruction scheduling, Architecture-specific optimizations, …
- DSLs separate algorithm description from performance implementation
- Modern AI compilers increasingly rely on domain-specific IRs

2.2. AI-DSL Design: How to Design DSL

- Tensor-Centric programming model
  - Matrix multiplication, Convolution, Reduction, Broadcast, Elementwise computation
- Declarative semantics
  - describe mathematical operations rather than execution order
    - C = A @ B
    - y = softmax(x)
- Hardware Independence
  - NVIDIA GPU, AMD GPU, CPU, TPU, Future accelerators
- Compiler optimizations enabled by DSLs
  - Operator fusion, Layout transformation, Automatic tiling, Parallel scheduling

2.3a. AI-DSL Design: Triton

- More friendly GPU development DSL
- Compiler Flow
  - Dialect->Triton IR->LLVM IR->PTX
- Compare with CUDA

	CUDA	TRITON
Memory Coalescing	Manual	Automatic
Shared Memory Management	Manual	Automatic
Scheduling (Within SMs)	Manual	Automatic
Scheduling (Across SMs)	Manual	Manual

- Optimizatio effect (softmax)

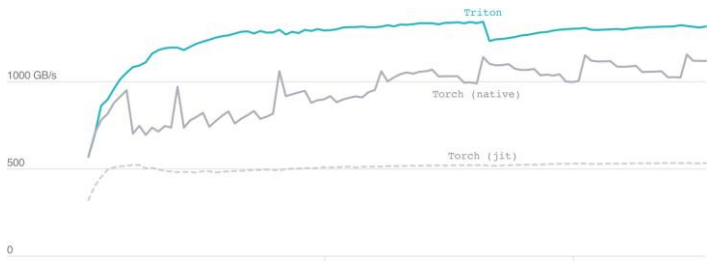


Image source: https://openai.com/index/triton/

```
@jit
def add(X, Y, Z, N):
  pid = program_id(0)
  idx = pid * 512 + arange(512)
  mask = idx < N
  x = load(X + idx, mask=mask)
  y = load(Y + idx, mask=mask)
  store(Z + idx, x + y, mask=mask)
```

```
def void add(i32* X .aligned(16),
    i32* Y .aligned(16),
    i32* Z .aligned(16),
    i32 N .multipleof(2)) {
entry:
  %0 = get_program_id[0] i32;
  %1 = mul i32 %0, 512;
  %3 = make_range[0 : 512] i32<512>;
  %4 = splat i32<512> %1;
  %6 = add i32<512> %4, %3;
  %9 = splat i32<512> N;
  %11 = icmp_slt i1<512> %6, %9;
  %14 = splat i32*<512> X;
  %16 = getelementptr i32*<512> %14, %6;
  %19 = broadcast i1<512> %11;
  %21 = splat i32<512> undef;
  %22 = masked_load i32<512> %16, %19, %21;
  %26 = splat i32*<512> Y;
  %28 = getelementptr i32*<512> %26, %6;
  %31 = broadcast i1<512> %11;
  %33 = splat i32<512> undef;
  %34 = masked_load i32<512> %28, %31, %33;
  %38 = splat i32*<512> Z;
  %40 = getelementptr i32*<512> %38, %6;
  %43 = add i32<512> %22, %34;
  %46 = broadcast i32<512> %43;
  %48 = broadcast i1<512> %11;
  masked_store void %40, %46, %48;
  ret void;}
```

```
.visible .entry add(
  .param .u64 add_param_0,
  .param .u64 add_param_1,
  .param .u64 add_param_2,
  .param .u32 add_param_3
)
.maxntid 128, 1, 1
{
  .reg .pred %p<4>;
  .reg .b32 %r<18>;
  .reg .b64 %rd<8>;
  ld.param.u64 %rd4, [add_param_0];
  ld.param.u64 %rd5, [add_param_1];
  mov.u32 %r13, %tid.x;
  ld.param.u32 %r14, [add_param_3];
  shl.b32 %r15, %r13, 2;
  mov.u32 %r16, %ctaid.x;
  mad.lo.s32 %r17, %r16, 512, %r15;
  setp.ge.s32 %p3, %r17, %r14;
  setp.lt.s32 %p1, %r17, %r14;
  mul.wide.s32 %rd7, %r17, 4;
  add.s64 %rd2, %rd4, %rd7;
  @%p1 ld.global.cg.v4.b32
    {%r5,%r6,%r7,%r8}, [%rd2 + 0];
  add.s64 %rd3, %rd5, %rd7;
  @%p1 ld.global.cg.v4.b32
    {%r9,%r10,%r11,%r12}, [%rd3 + 0];
  @%p3 bra LBB0_2;
  ld.param.u64 %rd6, [add_param_2];
  add.s64 %rd1, %rd6, %rd7;
  add.s32 %r1, %r5, %r9;
  add.s32 %r2, %r6, %r10;
  add.s32 %r3, %r7, %r11;
  add.s32 %r4, %r8, %r12;
  st.global.v4.u32
    [%rd1], {%r1, %r2, %r3, %r4};
LBB0_2:
  ret;
}
```

2.3b. AI-DSL Design: Triton

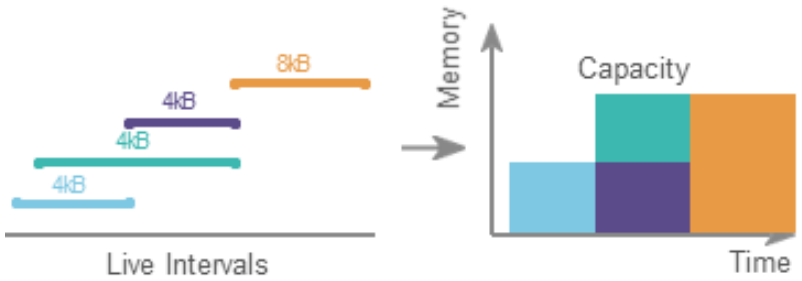
- Expose block-parallelism
  - Each kernel instance carry on block operation in parallel

```
BLOCK = 512
# This is a GPU kernel in Triton.
# Different instances of this
# function may run in parallel.
@jit
def add(X, Y, Z, N):
  pid = program_id(0)
  # block of indices
  idx = pid * BLOCK + arange(BLOCK)
  mask = idx < N
  ...
```

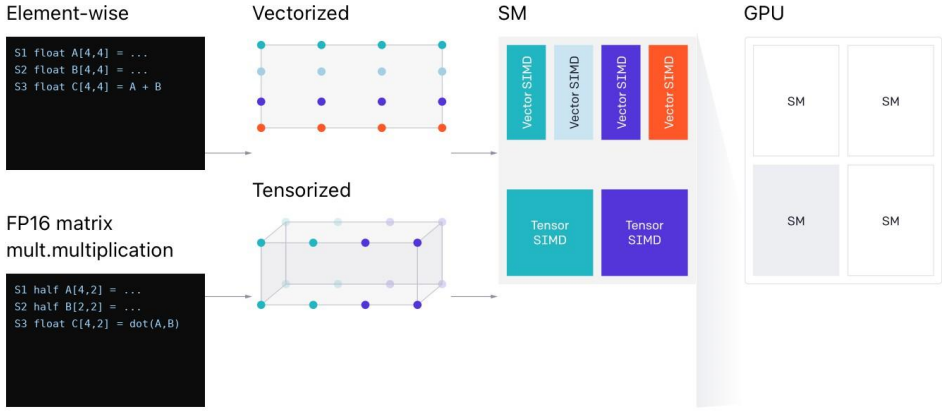
```
# Triton uses pointer
# arithmetics rather than
# indexing operators
x = load(X + idx, mask=mask)
y = load(Y + idx, mask=mask)
store(Z + idx, x + y,
      mask=mask)
```

```
...
grid = (ceil_div(N,
BLOCK),)
# no thread-block
add[grid](x, y, z,
          x.shape[0])
```

- Allocate shared memory based on varaible living analysis



- automatically parallelize opeartions both by instance and by SMs
  - Analyze the iteration space of each operation then partition



a. Definition of a Triton program *P* composed of three statements S1, S2, S3

b. Iteration space of S3

c. Mapping of S3 onto a Stream Multiprocessor (SM)

d. Mapping of *P* onto the GPU

Image source: https://openai.com/index/triton/

2.3c. AI-DSL Design: Triton

- Autotuning
  - Consider tile sizes
  - Explore at compilation

```
@triton.autotune(configs=[
    triton.Config(kwargs={'BLOCK_SIZE': 128}, num_warps=4),
    triton.Config(kwargs={'BLOCK_SIZE': 1024}, num_warps=8),
],
key=['x_size'] # the two above configs will be evaluated anytime
               # the value of x_size changes
)
@triton.jit
def kernel(x_ptr, x_size, **META):
    BLOCK_SIZE = META['BLOCK_SIZE']
```

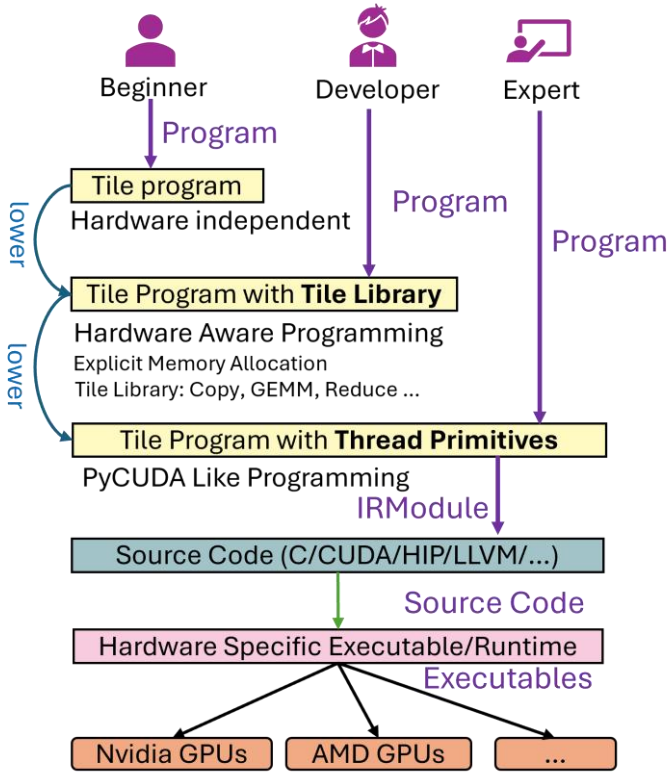
- Optimization passes
  - Computation graph patttern substitution
  - Dead code elimination

Image source: https://openai.com/index/triton/

- Use MLIR to lower layer by layer
- Most passes reuse MLIR
  - Arith: addf, addi, constant, cmpf, cmpi, …
  - Math: exp, sin, cos, log, absi, absf, sqrt
  - StructuredControlFlow: for, if, while, yield, condition
  - Reduce cost lowering to LLVM
- LLVM emits final GPU code

2.4a. AI-DSL Design: TileLang

- Provide three different programming interfaces
  - Beginner Level: Hardware-Unaware
  - Developer Level: Hardware-Aware with Tile Library
  - Expert Level: Hardware-Aware with Thread Primitives
- Compilation Flow
  - Tile Program: A high-level specification of the computation
  - Tile Program with Tile Library
  - Tile Program with Thread Primitives
  - IRModule
  - Source Code Generation (C/CUDA/HIP/LLVM/…)



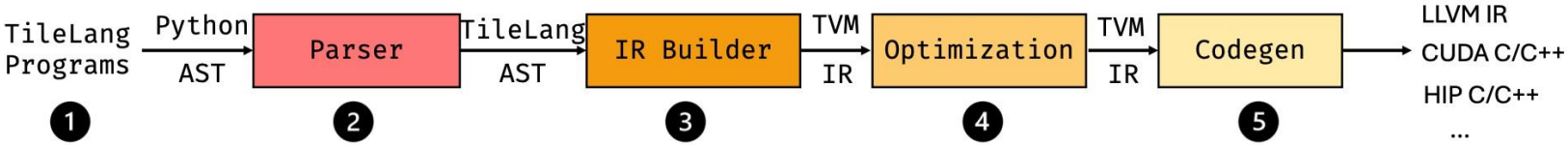
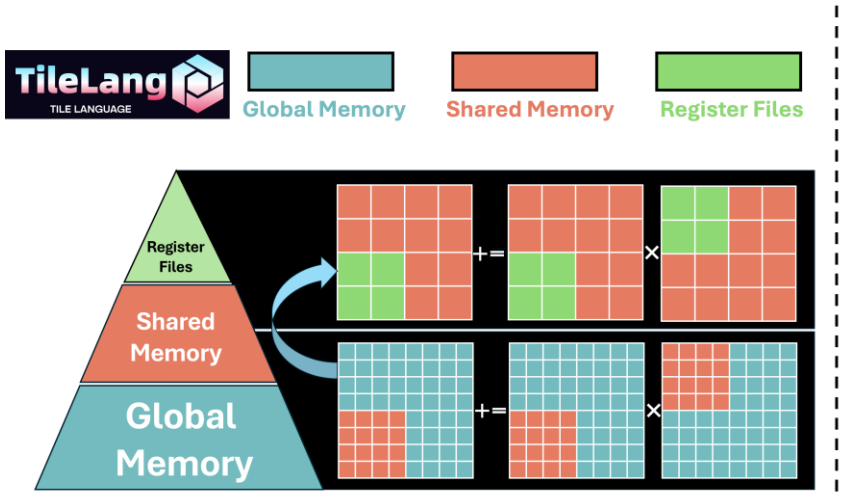


Image Source: https://arxiv.org/pdf/2504.17577v2 https://www.tilelang.com/get_started/overview.html

2.4b. AI-DSL Design: TileLang

- Tile declarations: Tiles as First-Class Objects
  - A tile represents a shaped portion of data
    - Can be owned and manipulated by a warp, thread block, or equivalent parallel unit
    - A and B buffers are read in tiled chunks (in example below)
  - Execution context: thread block index (bx and by) and the number of threads
    - Make it easier for the TileLang to automatically optimize



(a) Efficient GEMM with Multi-Level Tiling on GPUs

```
import tilelang.language as T

def Matmul(A: T.Buffer, B: T.Buffer, C: T.Buffer):
    Kernel Context Initialization
    with T.Kernel(
        ceildiv(N, block_N), ceildiv(M, block_M), threads=128
    ) as (bx, by):
        Buffer Allocation
        A_shared = T.alloc_shared((block_M, block_K))
        B_shared = T.alloc_shared((block_K, block_N))
        C_local = T.alloc_fragment((block_M, block_N), accum_dtype)
        T.clear(C_local)                    Initialize Accumulate Buffer with Zero
        Main Loop with Pipeline Annotation
        for k in T.Pipelined(ceildiv(K, block_K), num_stages=3):
            Copy Data from Global to Shared Memory
            T.copy(A[by * block_M, k * block_K], A_shared)
            T.copy(B[k * block_K, bx * block_N], B_shared)
            GEMM
            T.gemm(A_shared, B_shared, C_local)
        Write Back to Global Memory
        T.copy(C_local, C[by * block_M, bx * block_N])
```

Shared Memory
Register

(b) Describing Tiled GPU GEMM with TileLang

Image Source: https://arxiv.org/pdf/2504.17577v2

2.5a. AI-DSL: TileLang

- Explicit Hardware Memory Allocation:
  - Exposes user-facing intrinsics for tile buffers placement in the hardware memory hierarchy
  - Allocates memory in on-chip storage space
    - `T.alloc_shared`
    - Corresponds to shared memory on NVIDIA GPUs
  - Allocates accumulators in fragment memory
    - `T.alloc_fragment`
    - corresponds to register files on NVIDIA GPUs

- Explicit Hardware Memory Allocation:
  - `T.clear`: Initialize hardware-specific buffers
  - `T.fill`: Almost same as `T.clear`
  - `T.copy`: Data transfer between global memory and hardware-specific memory
  - `T.Parallel`: operations can also be performed in parallel for data assignments

2.5b. AI-DSL: TileLang

- Abstracts a Set of Tile Operators for Users to Use or Define

```
class TileOperator {
  Stmt Lower();
  LayoutMap InferLayout();
}
```

class GEMM	class Copy	class Parallel
T.GEMM(A_shared, B_shared, C_local)	T.Copy(A, A_shared)	for i, j in T.Parallel(128, 8): C_local[i, j] *= Scale[j]
T.call_extern("cute::gemm_ss", args)	for i in T.vectorized(8): A_shared[tx, i] = A[tx, i]	for i in T.vectorized(8): C_local[tx, i] += Scale[i]
AShared: MakeSwizzleLayout, BShared: MakeSwizzleLayout, CLocal: MakeMMASTMatrixLayout,	Loop Layout Lambda i, j->(tid, local_id)	Scale: InferencedScaleLayout,

- Each Tile Operator is required to implement two key interfaces
  - `Lower`: Defines how the high-level Tile Operator is lowered into a lower-level IR
  - `InferLayout`: Determine the memory and loop layouts associated with the Tile Operator
- Several Pre-defined Operators
  - `gemm`: highly optimized implementation for general matrix multiplication
  - `reduce`: flexible and efficient reduction mechanism for aggregating data across dimensions
  - `atomic`: atomic operations for safe updates to shared or global memory in a parallel context

Image Source: https://arxiv.org/pdf/2504.17577v2

2.6. TileLang: Schedule Annotations and Primitives

- TileLang provides a comprehensive suite of scheduling primitives
- `T.Pipelined`: allows efficient pipelined execution of loops to improve performance
  - Overlapping computation and memory operations
- `T.Parallel`: automatic parallelization of loops by mapping iterations to threads
- `T.annotate_layout`: specify memory layout optimizations for shared or global memory
- `T.use_swizzle`: Improves L2 cache locality by enabling swizzled memory accesses

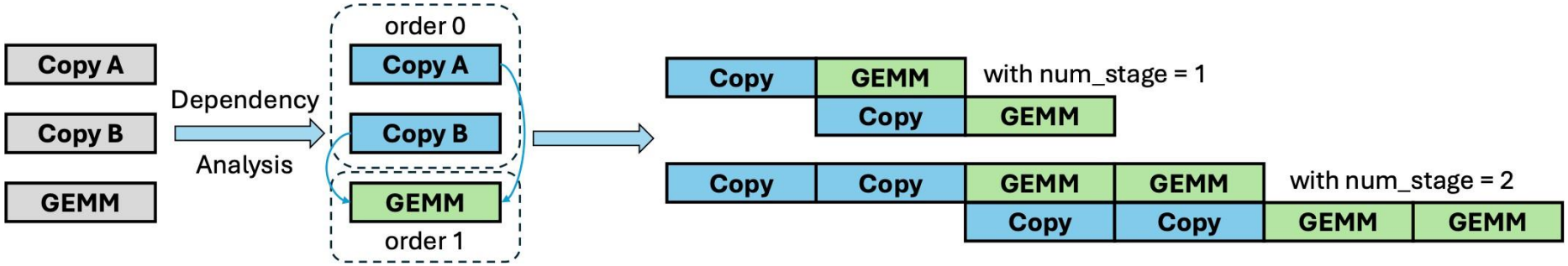


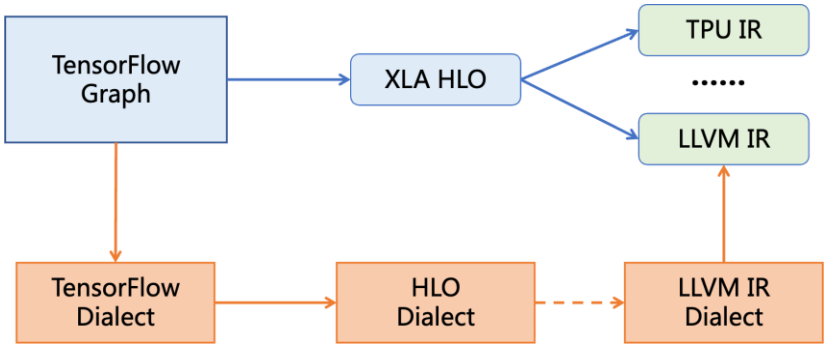
Image Source: https://arxiv.org/pdf/2504.17577v2

3. Optimization Passes

- Optimization Passes: MLIR Architecture
- Optimization Passes: MLIR Components
- MLIR Introduction
- Optimization Passes: TTIR
- Optimization Passes: TTGIR
- Optimization Passes: LLVM

3.1. Optimization Passes: MLIR Architecture

- MLIR A reusable, extensible open-source compilation infrastructure framework
  - Addressing software fragmentation
  - Support for compilation targeting heterogeneous hardware
  - Reducing overhead for domain-specific compiler development
  - Connecting existing various compilers
- MLIRWhy Can Solve the Problem
  - Unified syntax, reusable components
  - Shared ecosystem, coordinated optimization across levels
  - Small gaps between levels, easy to implement





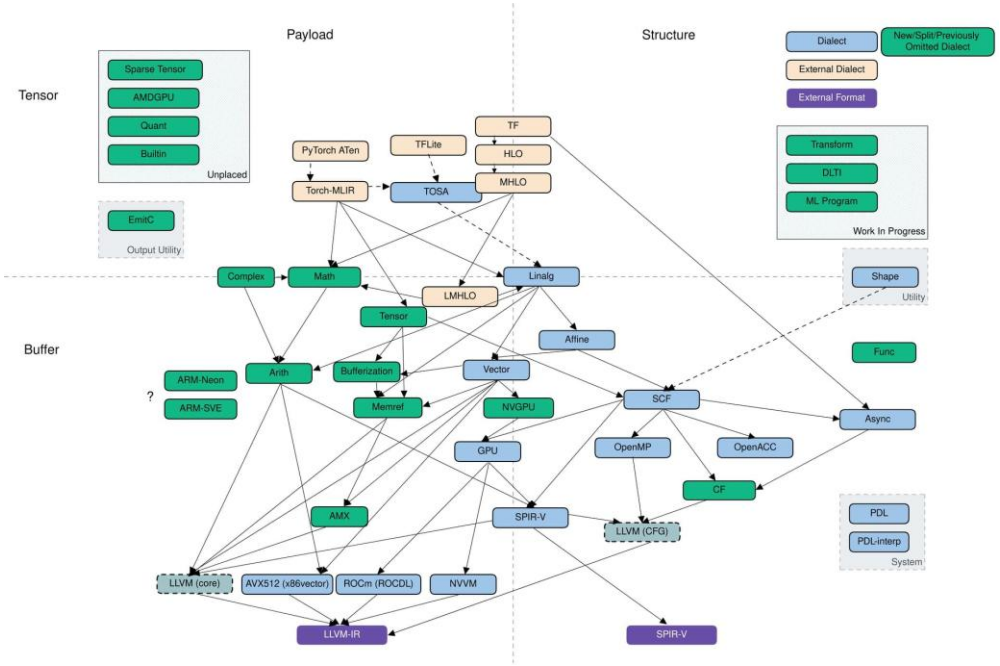
3.2. Optimization Passes: MLIR Components

- Module:
  - Top-level container encapsulating the entire compilation unit, isolating external variable scopes .
- Region:
  - Breaking through flat CFG limitations, preserving loops as “first-class citizens” in nested structures.
- Block:
  - Introducing Block Arguments avoiding traditional LLVM high-maintenance PHI nodes
- Operation and Attribute:
  - Highly unified opaque API, along with globally uniqued attribute dictionaries for representing constants.

```
%results:2 = "d.operation"(%arg0, %arg1) ({
  // Regions belong to Ops and can have multiple blocks.                    Region
 ^block(%argument: !d.type):                                               Block
   // Ops have function types (expressing mapping).
   %value = "nested.operation"() ({
     // Ops can contain nested regions.                                     Region
     "d.op"() : () -> ()
   }) : () -> (!d.other_type)
   "consume.value"(%value) : (!d.other_type) -> ()
 ^other_block:                                                             Block
   "d.terminator"() [^block(%argument : !d.type)] : () -> ()
 })
// Ops can have a list of attributes.
{attribute="value" : !d.type} : () -> (!d.type, !d.other_type)
```

3.3. MLIR Introduction

- MLIR: A Reusable Compilation Pipeline
  - Reusable IR、Pass
  - Serialize/De-serialize System
  - Multiple dailect can co-exist
- MLIR Dialect Translation Graph
  - Payload: Actual Operations (+-*/)
  - Structure: Control Flow; Function
  - Tensor: high level tensors
  - Buffer: low level pointers



3.4. Optimization Passes: TTIR

- Triton lowering process
  - Python Triton Kernel
  - Triton Frontend (AST + JIT specialization)
  - TTIR (Triton MLIR Dialect)
  - TTGIR (Triton GPU Dialect)
  - LLVM IR (NVVM Dialect)
  - PTX
  - SASS
- Tensor-Level optimization passes (TTIR)
- Canonicalization: simplifies IR by
  - constant folding
  - algebraic simplification
  - dead operation removal
  - redundant cast elimination
- Layout Propagation
  - blocked layouts
  - shared layouts
  - sliced layouts

3.5a. Optimization Passes: TTGIR

- GPU Mapping Passes
  - Lowering Tensor Programs onto GPU Execution Model
- TTGIR introduces GPU hardware concepts absent from TTIR
- Thread Mapping
  - Maps tensor tiles onto, programs, warps, lanes
- Shared Memory Promotion
  - Frequently reused tensors
  - Global memory->shared memory
- Software Pipelining
  - overlap memory latency
  - controlled by num_stages
  - Reschedule can help enhance efficiency

Load(k+1)
Compute(k)
Store(k−1)

- Block Tiling
  - MxN matrix -> BLOCK_M × BLOCK_N tiles
  - Specified/autotuned by python triton kernel

3.5b. Optimization Passes: TTGIR

- Warp-level optimization
  - warp shuffle
  - warp reduction
  - warp synchronization
- Memory coalescing
  - Transforms scattered accesses into contiguous memory transactions

Thread
0 -> A0
1 -> A64
2 -> A3

Thread
0 -> A0
1 -> A1
2 -> A2

- Vectorization
  - improving memory bandwidth
  - load f32->load.v4.f32
- Tensor core pattern recognition
  - Detects matrix multiply patterns
  - lower to mma.sync

3.6. Optimization Passes: LLVM

- LLVM no longer sees tensors
  - SSA values
  - basic blocks
  - scalar instructions
- mem2reg
  - Promotes stack variables into registers
- Instruction Combining
  - (a+b)+c -> a+(b+c)
  - mul(x,2) -> shift
- Global Value Numbering
  - Eliminates redundant computations across control flow
- Loop-Invariant code motion
  - Moves invariant computations outside loops
- Register allocation
  - Assigns virtual registers to physical GPU registers
- LLVM performs classical compiler optimizations independent of machine learning

3.7. Optimization Passes: Parameter and Effect

- Relating kernel parameters to compiler decisions

Parameter	Compiler Pass	Effect
BLOCK_M/N/K	Tile Formation	Tile size
num_warps	Thread Mapping	Warp decomposition
num_stages	Software Pipelining	Latency hiding
Grid	Program Scheduling	CTA generation
dtype	Instruction Selection	Tensor Core eligibility

African Scientific Research and Innovation Council

BAAI Beijing Academy of Artificial Intelligence

PEKING UNIVERSITY

RENMIN UNIVERSITY OF CHINA

Thanks

Questions & further discussion welcome

Guojie Luo

gluo@pku.edu.cn

Jing Wang

jwang@ruc.edu.cn

BAAI Team

github.com/flagos-ai

